RAMDEOBABA
UNIVERSITY

PROJECT TITLE

# CURE

## Clinical Urgency and Risk Evaluation: a role-controlled patient intake and triage workflow prototype


| | |
|---|---|
| **Presented by** | Huzaif Nazim Baig — Roll No. 58 |
| **Department / School** | Department of Computer Science & Engineering |
| **Project type** | B.Tech academic software prototype |


**Notice:** CURE is not a diagnostic system, a medical advice service, or a clinically validated product. Demonstrations use fictional information only.

# PROBLEM STATEMENT

## 1 What problem do we demonstrate?

Patient information, uploaded documents, and staff review can be spread across separate steps.

## 2 What CURE demonstrates

One role-controlled workflow for collecting a patient's answers, saving a screening snapshot, showing rule-based emergency warnings, routing a screening through triage, and recording a doctor's review.

## 3 Project focus

The project focuses on **connecting these software steps** and **controlling access to records**.

## 4 What CURE does not claim

- It is not a diagnostic system or a medical advice service.
- It is not a clinically validated product.
- It does not determine a diagnosis or the medically correct level of care.

**In a real emergency, seek emergency help — do not wait for CURE.**

# PROPOSED SOLUTION

WHAT CURE MEANS

## Clinical Urgency and Risk Evaluation

PROJECT IDEA

A prototype that shows how a patient intake record moves through **patient, triage, doctor and administrator** workspaces.

SCOPE

B.Tech academic software prototype. Not clinically validated.

### 1 Proposed approach

**Workflow:** patient answers the questionnaire → screening snapshot saved → rule-based emergency warning → triage routes to a doctor → doctor records a review.

**Access control:** a Python FastAPI backend validates requests, applies role and record-access checks, and runs the workflow operations.

### 2 Key design points

- **Screening snapshot:** responses, missing answers, screening status and relevant patient history are saved at submission time.
- **Rule-based emergency warning:** a separate set of configured rules checks a small set of emergency patterns — not an ML result or diagnosis.
- **Unverified extraction:** text extracted from uploaded reports is shown as unverified until the patient confirms the structured information.
- **Doctor review:** confirmation, or a correction recorded with a reason.

# OBJECTIVES

## 01

Collect patient intake: profile, fictional report upload with confirmed details, and the symptom questionnaire.

## 02

Save a screening snapshot: responses, missing answers, screening status and patient history at submission time.

## 03

Show rule-based emergency warnings from a small set of configured emergency patterns.

## 04

Route each screening through triage to a doctor and record the doctor's review, with access controlled by role.

WHAT THE PROTOTYPE COVERS

**4 roles**

Patient · Triage staff · Doctor · Administrator

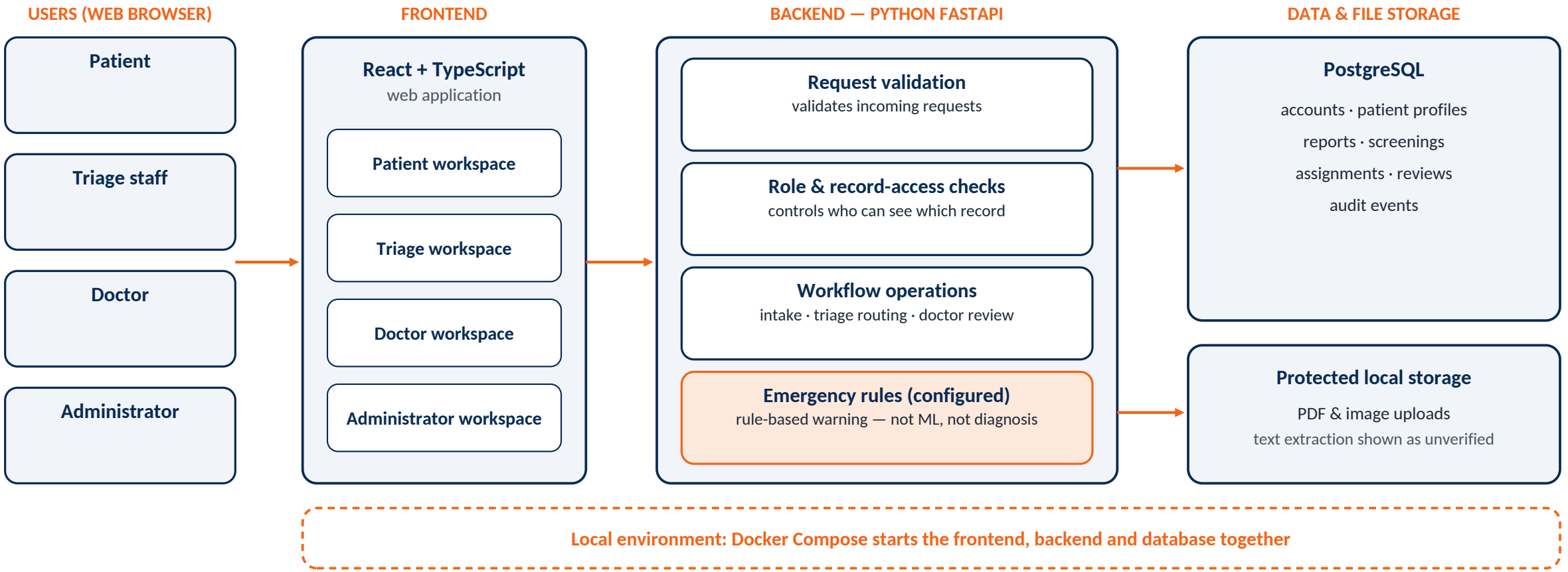
**1 workflow**

Intake → triage → doctor review

**No ML**

Warnings are rule-based; the ML experiment is separate

# SYSTEM ARCHITECTURE




**RESEARCH ML PIPELINE (SEPARATE EXPERIMENT)**

CDC NHAMCS survey data → compares logistic regression and random forest. Not loaded by the web application; does not predict a live patient's urgency.

**NOT CONNECTED**

live screenings show ML urgency unavailable

# SCREENING WORKFLOW

1 Register & sign in → 2 Profile & report upload → 3 Patient confirms details → 4 Questionnaire submitted → 5 Screening saved → 6 Triage routes to doctor → 7 Doctor review

## Patient steps

- Registers and signs in
- Updates profile, uploads a fictional report, reviews extracted text
- Confirms fictional structured details
- Answers the symptom questionnaire and submits

## On submission

- Saves responses, missing answers, screening status and a history snapshot
- Configured rules check emergency patterns; a match shows an **urgent warning** (rule-based, not ML)

## Triage & doctor

- Triage staff review the queue; emergency-rule flags appear for urgent human attention
- Triage routes a screening to a doctor
- Doctor confirms, or corrects with a reason

## Administrator

- Creates or manages accounts
- Views aggregate operational counts
- Not intended for routine access to clinical documents

# WHAT EACH ROLE CAN DO

## Patient

- Create an account with a new email address
- Complete profile and symptom questionnaire
- Upload a synthetic image or PDF sample
- Review extraction, verify fictional details, submit a screening
- View screening history

## Triage staff

- View the intake queue
- Start or complete intake
- Route a screening to a doctor
- Emergency-rule flags appear in the queue for urgent human attention

## Doctor

- View screenings for patients assigned to that doctor
- Review questionnaire responses and the screening-time history snapshot
- Confirm the review, or correct the urgency field and give a reason
- Access verified report information for assigned patients

## Administrator

- Create doctor and triage accounts with unique emails
- Change account roles or active status, subject to safeguards
- View aggregate operational analytics
- Not intended for routine access to clinical documents

# IMPLEMENTATION

### Frontend

React and TypeScript web application presenting the patient and staff workspaces.

### Backend

Python FastAPI service: validates requests, applies role and record-access checks, runs workflow operations.

### Database

PostgreSQL stores accounts, patient profiles, reports, screenings, assignments, reviews and audit events.

### Report handling

PDF and image uploads kept in protected local storage; extracted text is shown as unverified until the patient confirms it.

### Run locally (Docker Compose)

```
docker compose up --build -d

docker compose ps

GET http://localhost:8000/api/v1/health

docker compose exec backend python -m app.bootstrap_admin

App:       http://localhost:5173

API docs:  http://localhost:8000/docs

docker compose down
```

Initial admin is created once; if the email already exists it has already been created.

**LOCAL CONFIGURATION**

Demo-only values in a local **.env** file: POSTGRES_PASSWORD, JWT_SECRET_KEY, INITIAL_ADMIN_EMAIL, INITIAL_ADMIN_PASSWORD (12+ characters). Not shared or committed. **docker compose down** keeps the database and uploads; avoid **--volumes** unless deleting them on purpose.

# LIVE DEMONSTRATION

## Suggested flow (synthetic accounts and files only)

1. Sign in as the administrator from the local .env; create one doctor and one triage-staff account.
2. Register a fictional patient with a new email and a password of at least 12 characters.
3. Follow Case A: enter the listed fictional answers, upload the matching sample image, verify the extracted content and submit.
4. Optional warning path: use Case B or C and its matching image.
5. Sign in as triage staff, open the screening and route it to the doctor (triage asks for the doctor's account ID).
6. Sign in as that doctor, open the assigned screening and record a review.
7. Return to the administrator to show account management and aggregate analytics.

*Refresh the browser to return to the sign-in screen when changing roles.*

### SYNTHETIC CASES

**Case A** — saves without an emergency warning; its ML result remains unavailable.

**Case B / Case C** — demonstrate the warning path. The message comes from the configured emergency rules.

These show the workflow and the separate warning rules, not live model inference.

### Demo ground rules

- Fictional information only
- Each email must be unused in the local database
- New account passwords: 12+ characters

# RESEARCH ML STATUS

### 1 Experiment

A research-only training/evaluation pipeline compared **logistic regression** and **random forest**.

### 2 Data & labels

**2022 CDC NHAMCS** Emergency Department public-use data. Labels are historical provider-recorded triage categories, not outcomes that establish CURE's clinical accuracy.

### 3 Result

Recorded results were **weak for rare classes**.

LIVE SCREENINGS

## ML urgency unavailable

The ML model is **not integrated** with the application and is not loaded by the web app.

Emergency warnings come from **separate configured rules**, not from the model.

**Model results must not be presented as clinical predictions.**

# VERIFICATION & REMAINING WORK

**18**

backend integration tests passing

**Build**

frontend production build successful

**Healthy**

PostgreSQL service

**ok / 200**

API health response / frontend HTTP status

## Also recorded

- Synthetic PDF, PNG and JPG samples were reported to produce non-empty text extraction output.
- A full authenticated role-by-role browser walkthrough remains a manual check.

## Work still needed beyond the prototype

- A qualified clinician must review the emergency rules before any real-patient use.
- The research ML model needs independent clinical validation before any integration is considered.
- Production privacy, retention, security and deployment operations need review.

# CONCLUSION

## Summary

CURE is an academic prototype for organizing a patient intake and staff review workflow.

Patients enter fictional intake information and can upload reports; the system saves a screening snapshot and applies a separate set of emergency-warning rules. Triage staff route screenings to a doctor, who records a review, while an administrator manages accounts and views aggregate information.

We tested the software workflow, but CURE is **not clinically validated**.

### What we do not claim

- The app does not diagnose patients.
- The app does not predict clinical outcomes.
- The ML experiment is separate and not connected to live predictions.
- Emergency warnings are rule-based and need clinician review before real-patient use.